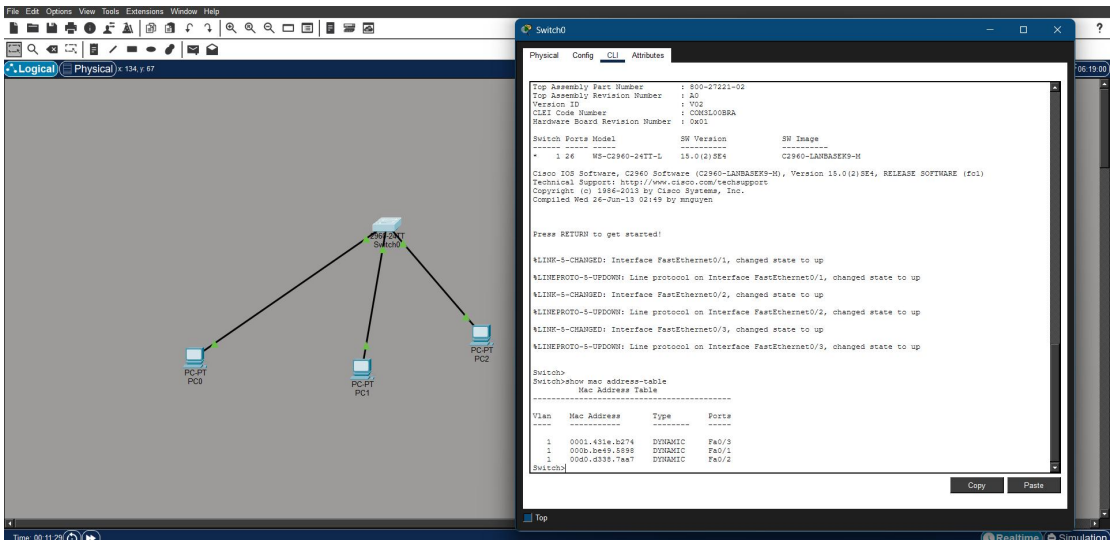
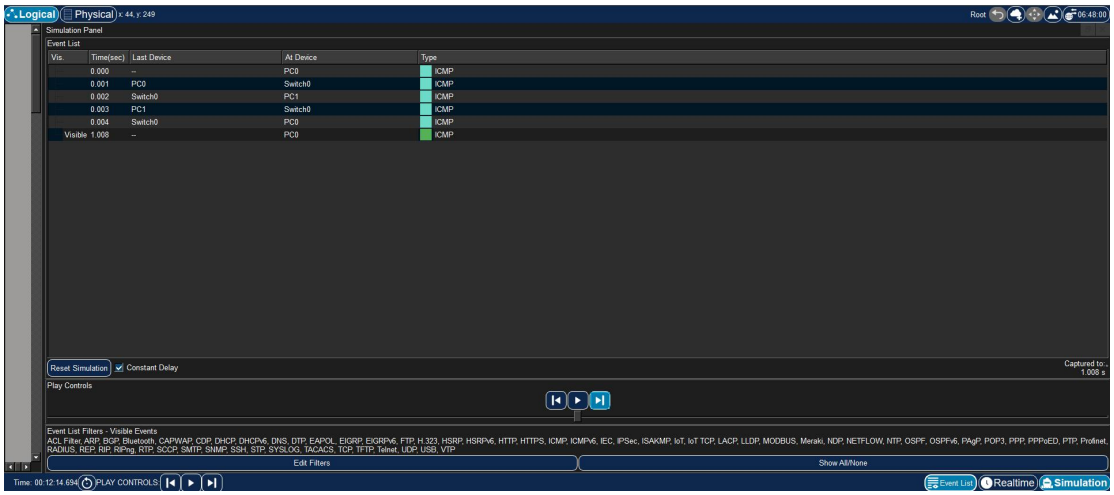
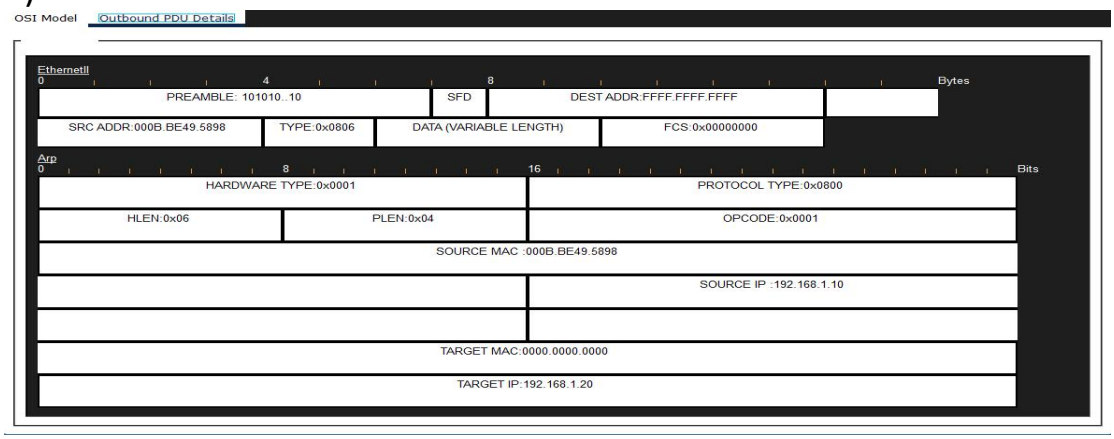
1)

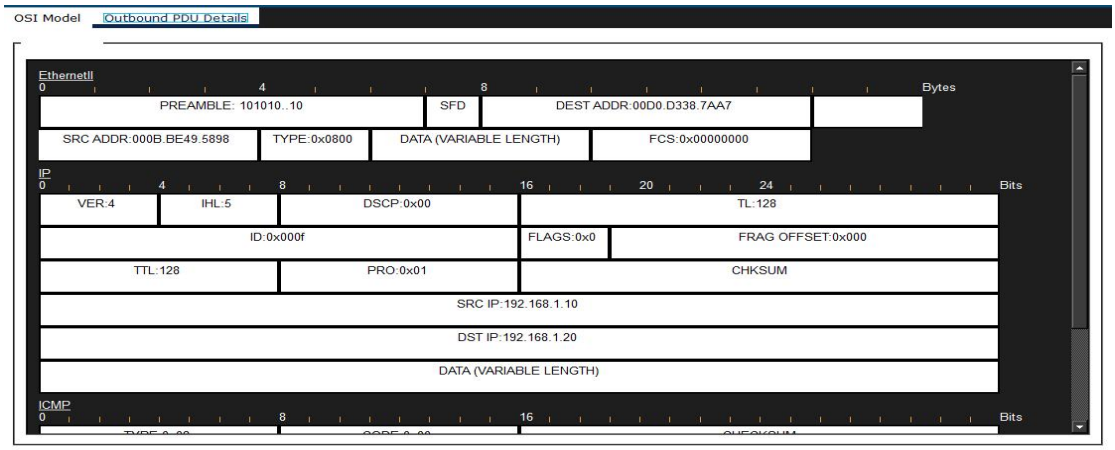

File Edit Options View Tools Extensions Window Help
Logical
Physical
Switch0
Physical
Config
CLI
Attributes
Top Assembly Part Number : 800-27221-02
Top Assembly Revision Number : A0
Version ID : V02
CLEI Code Number : CONSL00BRA
Hardware Board Revision Number : 0x01
Switch Ports Model SW Version SW Image
* 1 26 WS-C2960-24TT-L 15.0(2)SE4 C2960-LANBASEK9-M
Cisco IOS Software, C2960 Software (C2960-LANBASEK9-M), Version 15.0(2)SE4, RELEASE SOFTWARE (fc1)
Technical Support: http://www.cisco.com/techsupport
Copyright (c) 1986-2013 by Cisco Systems, Inc.
Compiled Wed 26-Jun-13 02:49 by mnguyen
Press RETURN to get started!
%LINK-5-CHANGED: Interface FastEthernet0/1, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/1, changed state to up
%LINK-5-CHANGED: Interface FastEthernet0/2, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/2, changed state to up
%LINK-5-CHANGED: Interface FastEthernet0/3, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/3, changed state to up
Switch>
Switch>show mac address-table
Mac Address Table
Vlan Mac Address Type Ports
1 0001.43le.b274 DYNAMIC Fa0/3
1 000b.be49.5898 DYNAMIC Fa0/1
1 00d0.d338.7aa7 DYNAMIC Fa0/2
Switch>
Copy
Paste
Top
PC-PT
PC0
PC-PT
PC1
PC-PT
PC2
Realtime
Simulation


Logical
Physical
Simulation Panel
Event List
Vis. Time(sec) Last Device At Device Type
0.000 -- PC0 ICMP
0.001 PC0 Switch0 ICMP
0.002 Switch0 PC1 ICMP
0.003 PC1 Switch0 ICMP
0.004 Switch0 PC0 ICMP
Visible 1.008 -- PC0 ICMP
Reset Simulation
Constant Delay
Play Controls
Event List Filters - Visible Events
Edit Filters
Show All/None
Event List
Realtime
Simulation

2)


OSI Model
Outbound PDU Details
EthernetII
Bytes
PREAMBLE: 101010...10
SFD
DEST ADDR:FFFF.FFFF.FFFF
SRC ADDR:000B.BE49.5898
TYPE:0x0806
DATA (VARIABLE LENGTH)
FCS:0x00000000
Arp
Bits
HARDWARE TYPE:0x0001
PROTOCOL TYPE:0x0800
HLEN:0x06
PLEN:0x04
OPCODE:0x0001
SOURCE MAC :000B.BE49.5898
SOURCE IP :192.168.1.10
TARGET MAC:0000.0000.0000
TARGET IP:192.168.1.20

- To complete this task, I used Cisco Packet Tracer's Simulation Mode to capture and analyse Ethernet frames between PC0 (192.168.1.10) and PC1 (192.168.1.20) connected through a 2960 switch. The first frame captured was an ARP Request. The Destination MAC was FFFF.FFFF.FFFF (broadcast), Source MAC was 000B.BE49.5898 (PC0), and the Ethertype was 0x0806 identifying it as an ARP packet. The ARP payload showed Opcode 0x0001 (Request), Source IP 192.168.1.10, Target IP 192.168.1.20, and Target MAC 0000.0000.0000 because PC0 did not yet know PC1's MAC address. Since it was a broadcast, the switch flooded this frame to all ports and only PC1 responded.



- After PC0 learned PC1's MAC address from the ARP reply, it sent an ICMP Echo Request frame. This time the Destination MAC was 00D0.D338.7AA7 (PC1's actual MAC), Source MAC was 000B.BE49.5898 (PC0), and the Ethertype was 0x0800 identifying it as an IPv4 packet. The IP payload showed Source IP 192.168.1.10, Destination IP 192.168.1.20, TTL 128, and Protocol 0x01 confirming ICMP. Since the destination MAC was now a specific unicast address, the switch forwarded the frame directly to PC1's port only without any flooding. This task demonstrated how ARP resolves MAC addresses before communication begins and how the Ethertype, MAC addresses, and FCS fields each play a specific role in Ethernet frame delivery.

3)

```
nivesh@LAPTOP-8KUN9AG2:~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.30.122.106  netmask 255.255.240.0  broadcast 172.30.127.255
        inet6 fe80::215:5dff:fe66:2c79  prefixlen 64  scopeid 0x20<link>
        ether 00:15:5d:66:2c:79  txqueuelen 1000  (Ethernet)
        RX packets 347  bytes 266554 (266.5 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 204  bytes 18352 (18.3 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 26  bytes 3134 (3.1 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 26  bytes 3134 (3.1 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

nivesh@LAPTOP-8KUN9AG2:~$ sudo ip addr add 192.168.1.10/24 dev eth0
nivesh@LAPTOP-8KUN9AG2:~$ sudo ip addr add 192.168.1.10/24 dev eth0
RTNETLINK answers: File exists
nivesh@LAPTOP-8KUN9AG2:~$ ip addr show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:15:5d:66:2c:79 brd ff:ff:ff:ff:ff:ff
    inet 172.30.122.106/20 brd 172.30.127.255 scope global eth0
       valid_lft forever preferred_lft forever
    inet 192.168.1.10/24 scope global eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::215:5dff:fe66:2c79/64 scope link
       valid_lft forever preferred_lft forever
nivesh@LAPTOP-8KUN9AG2:~$ sudo ip link set eth0 down
nivesh@LAPTOP-8KUN9AG2:~$ sudo ip link set eth0 address 02:AA:BB:CC:DD:10
nivesh@LAPTOP-8KUN9AG2:~$ sudo ip link set eth0 up
nivesh@LAPTOP-8KUN9AG2:~$ ip link show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP mode DEFAULT group default qlen 1000
    link/ether 02:aa:bb:cc:dd:10 brd ff:ff:ff:ff:ff:ff permaddr 00:15:5d:66:2c:79
```

```
nivesh@LAPTOP-8KUN9AG2:~$ ping -c 4 8.8.8.8
ping: connect: Network is unreachable
nivesh@LAPTOP-8KUN9AG2:~$ ping -c 4 127.0.0.1
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.037 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.041 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.027 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.026 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3089ms
rtt min/avg/max/mdev = 0.026/0.032/0.041/0.006 ms
nivesh@LAPTOP-8KUN9AG2:~$ ifconfig eth0
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.30.122.106  netmask 255.255.240.0  broadcast 172.30.127.255
        inet6 fe80::aa:bbff:fecc:dd10  prefixlen 64  scopeid 0x20<link>
        ether 02:aa:bb:cc:dd:10  txqueuelen 1000  (Ethernet)
        RX packets 397  bytes 287090 (287.0 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 226  bytes 19836 (19.8 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

4)

```
C:\>ping 192.168.1.10

Pinging 192.168.1.10 with 32 bytes of data:

Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.168.1.10:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),

C:\>tracert 192.168.1.10

Tracing route to 192.168.1.10 over a maximum of 30 hops:

  1   *         *         *         Request timed out.
  2   *         *         *         Request timed out.
  3   *         *
Control-C
^C
C:\>ping 192.168.1.10

Pinging 192.168.1.10 with 32 bytes of data:

Reply from 192.168.1.10: bytes=32 time<1ms TTL=128
Reply from 192.168.1.10: bytes=32 time<1ms TTL=128
Reply from 192.168.1.10: bytes=32 time<1ms TTL=128
Reply from 192.168.1.10: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.1.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>
```

5)

Linux Kernel Ethernet Support

The Linux kernel supports Ethernet communication through a layered networking subsystem. According to kernel.org, every network interface is represented by a data structure called struct net_device defined in include/linux/netdevice.h, which stores the interface name, MAC address, MTU size, and operational state flags like UP and RUNNING. When an Ethernet frame arrives, the kernel allocates a socket buffer called sk_buff, passes it up the stack using netif_receive_skb(), and inspects the Ethertype field to route it to the correct protocol handler such as ip_rcv() for IPv4 or arp_rcv() for ARP. For outgoing frames, the kernel builds the sk_buff, attaches the correct MAC addresses, and calls ndo_start_xmit() to pass it to the hardware. NAPI improves performance at high speeds by switching from interrupt mode to batch polling, reducing CPU overhead significantly. The kernel also exposes NIC settings and statistics through the ethtool interface for monitoring and configuration.

6)

```
nivesh@LAPTOP-8KUN9AG2:~$ ifconfig eth0
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.30.122.106  netmask 255.255.240.0  broadcast 172.30.127.255
        inet6 fe80::aa:bbff:fecc:dd10  prefixlen 64  scopeid 0x20<link>
        ether 02:aa:bb:cc:dd:10  txqueuelen 1000  (Ethernet)
        RX packets 397  bytes 287090 (287.0 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 226  bytes 19836 (19.8 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

nivesh@LAPTOP-8KUN9AG2:~$ ip addr show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 02:aa:bb:cc:dd:10 brd ff:ff:ff:ff:ff:ff permaddr 00:15:5d:66:2c:79
    inet 172.30.122.106/20 brd 172.30.127.255 scope global eth0
       valid_lft forever preferred_lft forever
    inet 192.168.1.10/24 scope global eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::aa:bbff:fecc:dd10/64 scope link
       valid_lft forever preferred_lft forever
nivesh@LAPTOP-8KUN9AG2:~$ sudo ip addr add 192.168.56.10/24 dev eth0
[sudo] password for nivesh:
nivesh@LAPTOP-8KUN9AG2:~$ ip addr show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 02:aa:bb:cc:dd:10 brd ff:ff:ff:ff:ff:ff permaddr 00:15:5d:66:2c:79
    inet 172.30.122.106/20 brd 172.30.127.255 scope global eth0
       valid_lft forever preferred_lft forever
    inet 192.168.1.10/24 scope global eth0
       valid_lft forever preferred_lft forever
    inet 192.168.56.10/24 scope global eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::aa:bbff:fecc:dd10/64 scope link
       valid_lft forever preferred_lft forever
nivesh@LAPTOP-8KUN9AG2:~$ ip route show
172.30.112.0/20 dev eth0 proto kernel scope link src 172.30.122.106
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.10
192.168.56.0/24 dev eth0 proto kernel scope link src 192.168.56.10
nivesh@LAPTOP-8KUN9AG2:~$ sudo ip link set eth0 down
nivesh@LAPTOP-8KUN9AG2:~$ sudo ip link set eth0 up
nivesh@LAPTOP-8KUN9AG2:~$ ip link show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP mode DEFAULT group default qlen 1000
    link/ether 02:aa:bb:cc:dd:10 brd ff:ff:ff:ff:ff:ff permaddr 00:15:5d:66:2c:79
nivesh@LAPTOP-8KUN9AG2:~$
```

Summary:

To configure a basic LAN interface in Linux, I used the ip command which is the modern replacement for ifconfig as documented at kernel.org. I ran ip addr show eth0 to view the current interface state which showed the MAC address, existing IP address, and that the interface was UP. I then assigned a new static IP address using sudo ip addr add

192.168.56.10/24 dev eth0 and verified it was added successfully by running ip addr show eth0 again which showed three active IP addresses on the same interface. Finally I confirmed the routing table using ip route show which automatically showed new network routes for each assigned IP, and restarted the interface using sudo ip link set eth0 down and sudo ip link set eth0 up to confirm full connectivity was restored.

7)

```
nivesh@LAPTOP-8KUN9AG2:~$ ip link show master br0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq master br0 state UP mode DEFAULT group default qlen 1000
    link/ether 02:aa:bb:cc:dd:10 brd ff:ff:ff:ff:ff:ff permaddr 00:15:5d:66:2c:79
nivesh@LAPTOP-8KUN9AG2:~$
```

```
nivesh@LAPTOP-8KUN9AG2:~$ ip route show
172.30.112.0/20 dev eth0 proto kernel scope link src 172.30.122.106
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.10
192.168.56.0/24 dev eth0 proto kernel scope link src 192.168.56.10
nivesh@LAPTOP-8KUN9AG2:~$
```

Conclusion:
Ethernet communication in a LAN is achieved through ARP-based address resolution and MAC based frame forwarding, while the Linux kernel efficiently manages packet processing through its networking stack.